

Project Instructions for Claude

Project

D2C artificial/imitation jewellery e-commerce website.

Primary Goal

Build a focused, production-quality MVP for selling artificial jewellery online.

Engineering Principle

Do not use vibe coding.

Always follow:

Requirement → User Flow → Data Model/API Contract → Architecture → Implementation → Testing

Before Coding

- Read the relevant docs.
- Identify the exact feature being implemented.
- Do not implement unspecified features.
- If a requirement is ambiguous and materially affects architecture/data/security, ask before coding.
- Prefer the smallest correct implementation.

Technology

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- TanStack Query
- Zustand
- React Hook Form
- Zod
- PostgreSQL
- Prisma
- Auth.js/equivalent
- Razorpay

- Cloudinary
- Vercel

State Rules

- TanStack Query: server state.
- Zustand: small client state such as cart/UI state.
- React Hook Form: form state.
- Do not duplicate server state unnecessarily.

Backend Rules

- Business logic must not live in UI components.
- Validate all external input.
- Server calculates price, stock, shipping, discount, and final total.
- Never trust client payment status.
- Protect admin operations server-side.

Product Rules

- This is artificial/imitation jewellery.
- Never make unsupported claims that products are real gold/silver.
- Product information must be accurate and transparent.

Scope Rules

MVP first. Do not add AI, advanced analytics, loyalty, referrals, multi-vendor, international payments, or other non-MVP functionality unless explicitly requested.

Code Quality

- Prefer readable code over clever abstractions.
- Avoid premature generic frameworks/utilities.
- Keep modules focused.
- Reuse components when there is a real repeated pattern.
- Do not create abstractions for one-time code without a reason.

Testing

Important business logic must have tests. Critical customer and admin journeys should have E2E coverage.

Change Protocol

If a requirement changes: 1. Explain the impact. 2. Update the relevant specification. 3. Then modify the code.

Never silently change architecture or product scope.

Product Specification --- Artificial Jewellery E- Commerce

1. Product Goal

Build a D2C e-commerce website for selling artificial/imitation jewellery under the brand's own identity.

The business will source artificial jewellery from wholesale markets, sell it under its own brand, and fulfil customer orders through the website.

The website must be predictable, maintainable, scalable, and intentionally scoped. No feature should be built without a defined product requirement.

2. Important Product Positioning

- Products are artificial/imitation jewellery.
- The website must never represent artificial jewellery as real gold, real silver, or another precious metal unless that claim is factually true for the specific product.
- Product material/specification information must be accurate and transparent.

3. Primary Customer Flow

Home → Product Listing → Product Details → Cart → Checkout → Address → Payment → Order Confirmation → Order Tracking.

4. MVP Goals

Customers can: - Browse products - Browse categories - Search products - View product details and images - Select available variants/quantity - Add/remove/update cart items - Enter/manage delivery address - Checkout - Pay online - View orders and order status - Create/login to an account

Admin can: - Manage products - Manage categories - Manage inventory/stock - View and manage orders - View customers

5. MVP Non-Goals

Do not build initially: - Mobile app - Multi-vendor marketplace - International shipping - Multi-currency - Loyalty points - Referral program - AI chatbot - AI recommendations - Advanced analytics - Subscription system - Complex coupon engine

These can be considered only after a real product requirement exists.

6. Product Categories

Initial categories: - Earrings - Necklace - Necklace Sets - Bangles - Rings - Bracelets - Other

7. Product Information

A product may contain: - Name - Slug - Description - Category - Price - Compare-at price - Images - Stock quantity - Material - Colour - Size - Weight, if applicable - Care instructions - Active/inactive status

Only information actually known for a product should be published.

8. Order Lifecycle

PENDING → CONFIRMED → PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

Other states: - PAYMENT_FAILED - CANCELLED - RETURN_REQUESTED - RETURNED - REFUNDED

9. Core Principle

Requirement → User Flow → Data Model → API Contract → UI → Implementation → Testing.

Do not implement features based on assumptions or coding convenience.

Technical Architecture

1. Architecture Goal

Use a TypeScript-first architecture that is simple for the MVP but can scale without a major rewrite.

2. Technology Stack

Application

- Next.js
- TypeScript

UI

- Tailwind CSS
- shadcn/ui

Client/Data

- TanStack Query for server state
- Zustand for small client-side state such as cart/UI state
- React Hook Form for forms
- Zod for validation

Backend

- Next.js Route Handlers
- TypeScript
- Service layer
- Repository/data-access layer where useful

Database

- PostgreSQL
- Prisma ORM

Authentication

- Auth.js or an equivalent well-supported Next.js authentication solution

Payments

- Razorpay

Images

- Cloudinary

Deployment

- Vercel for the Next.js application
- Managed PostgreSQL provider

3. High-Level Architecture

Browser → Next.js UI → TanStack Query / client state → Route Handlers → Services → Prisma → PostgreSQL

External integrations: - Razorpay for payments - Cloudinary for product images - Email provider only when a concrete email requirement is introduced

4. State Management Rules

Server state: - Products - Categories - Orders - Customer data

Use TanStack Query.

Client state: - Cart - Temporary UI state

Use Zustand where needed.

Form state: - React Hook Form.

Do not duplicate server state into global client state without a clear reason.

5. Architecture Rules

- Keep business logic out of React components.
- Validate external input with Zod.
- Keep payment verification on the server.
- Do not trust client-side prices, totals, stock, or payment status.
- Keep database access behind a predictable data-access boundary.
- Prefer small, focused modules over large generic utility files.
- Avoid premature abstractions.

6. Suggested Project Structure

```
src/ ├── app/ ├── components/ ├── features/ | ├── products/ | ├── cart/ | ├── checkout/ | ├── orders/ | └──  
    account/ ├── lib/ ├── hooks/ ├── services/ ├── store/ ├── types/ └── validations/
```

Backend logic should be organised by business capability rather than one giant services file.

Data Model Specification

1. Core Entities

User

- id
- name
- email
- phone
- password/auth provider data as required by the authentication solution
- role
- createdAt
- updatedAt

Roles: - USER - ADMIN

Category

- id
- name
- slug
- description
- isActive
- createdAt
- updatedAt

Product

- id
- name
- slug
- description
- categoryId
- price
- compareAtPrice
- material
- colour
- size
- weight

- careInstructions
- stockQuantity
- isActive
- createdAt
- updatedAt

ProductImage

- id
- productId
- url
- altText
- sortOrder

Address

- id
- userId
- fullName
- phone
- addressLine1
- addressLine2
- city
- state
- postalCode
- country
- isDefault

Cart

- id
- userId/session identifier as required
- createdAt
- updatedAt

CartItem

- id
- cartId
- productId
- quantity
- unitPrice snapshot where required

Order

- id
- userId
- status
- paymentStatus
- subtotal
- shippingAmount
- discountAmount
- totalAmount
- shippingAddress snapshot
- createdAt
- updatedAt

OrderItem

- id
- orderId
- productId
- productName snapshot
- quantity
- unitPrice
- totalPrice

Payment

- id
- orderId
- provider
- providerOrderId
- providerPaymentId
- amount
- status
- createdAt
- updatedAt

2. Important Data Rules

- Store monetary values safely and consistently; do not use floating-point arithmetic for financial calculations.
- Order items should preserve product name/price snapshots so historical orders remain correct if a product changes later.
- Stock must be validated server-side.
- Product and category deletion should normally be soft deactivation when historical references exist.
- Database constraints and indexes should be defined intentionally.

3. Relationship Overview

User — Addresses — Cart — Orders — Account/Auth data

Category — Products — ProductImages

Cart — CartItems — Product

Order — OrderItems — Payment — User

API Specification

1. API Principles

- REST-style HTTP APIs.
- JSON request/response format.
- Validate all incoming data.
- Authentication and authorization enforced server-side.
- Never trust client-calculated totals or payment status.
- Use consistent error responses.

2. Product APIs

GET /api/products - List products - Pagination - Category filter - Search - Sorting where required

GET /api/products/:slug - Product details

POST /api/admin/products - Create product

PATCH /api/admin/products/:id - Update product

DELETE /api/admin/products/:id - Deactivate product

3. Category APIs

GET /api/categories POST /api/admin/categories PATCH /api/admin/categories/:id DELETE /api/admin/categories/:id

4. Cart APIs

GET /api/cart POST /api/cart/items PATCH /api/cart/items/:id DELETE /api/cart/items/:id

The server must re-check: - Product existence - Product active status - Stock - Current price

5. Checkout APIs

POST /api/checkout - Validate cart - Validate address - Recalculate totals - Create internal order/payment intent as appropriate

6. Payment APIs

POST /api/payments/create POST /api/payments/verify POST /api/payments/webhook

Payment webhook/verification must be handled server-side.

7. Order APIs

GET /api/orders GET /api/orders/:id

Admin: GET /api/admin/orders PATCH /api/admin/orders/:id/status

8. Authentication

Authentication endpoints depend on the selected authentication provider/solution.

Required capabilities: - Register/sign in - Sign out - Session - Protected customer routes - Protected admin routes

9. Error Shape

Use one predictable structure:

```
{ "error": { "code": "PRODUCT_NOT_FOUND", "message": "Product not found" } }
```

Do not expose internal database errors or secrets to clients.

UI / UX Specification

1. Design Goal

Premium-looking artificial jewellery store with a clean Indian D2C e-commerce experience.

The UI should communicate: - Product quality - Trust - Clear pricing - Easy shopping - Transparent product information

2. Core Pages

/ - Header - Hero - Featured categories - Featured products - Best sellers - Trust/shipping/returns information - Footer

/products - Search - Category filters - Product grid - Pagination - Empty state - Loading state

/products/[slug] - Image gallery - Product name - Price - Product information - Variant selection if applicable - Quantity - Stock status - Add to cart - Buy now - Delivery information - Return information - Reviews when review functionality is implemented

/cart - Items - Quantity controls - Remove - Price summary - Checkout CTA - Empty cart state

/checkout - Customer information - Delivery address - Order summary - Payment

/order/success - Order confirmation - Order number - Summary - View order CTA

/orders - Customer order list

/orders/[id] - Order details - Status - Items - Address summary

/account - Profile - Addresses - Orders

/admin - Dashboard - Products - Categories - Orders - Customers

3. UX Rules

Every important state must have a defined UI: - Loading - Empty - Error - Success - Out of stock - Invalid input - Payment failure - Order failure

Do not hide important product/material information.

4. Responsive Design

Mobile-first. Must support: - Mobile - Tablet - Desktop

5. Accessibility

- Semantic HTML
- Keyboard navigation
- Visible focus states
- Accessible form labels
- Meaningful image alt text
- Sufficient contrast

Security & Testing Specification

Security

Authentication

- Protect customer account routes.
- Protect all admin routes.
- Enforce authorization server-side.

Input Validation

Validate: - Request bodies - Query parameters - Route parameters - Checkout data - Admin inputs

Use Zod.

Payment Security

- Never trust client payment success.
- Verify payment on the server.
- Validate amount/order relationship.
- Handle webhook events safely and idempotently.
- Never store sensitive card data.

Secrets

Keep secrets in environment variables. Never commit: - API keys - Payment secrets - Database credentials - Authentication secrets

E-commerce Integrity

Server must calculate: - Product price - Quantity - Stock - Shipping - Discount - Final total

Testing Strategy

Unit Tests

Test: - Price calculations - Cart calculations - Validation - Order status rules - Stock rules

Integration Tests

Test: - Product APIs - Cart APIs - Checkout - Order creation - Payment verification

E2E Tests

Critical user journey:

Home → Product → Cart → Checkout → Payment test flow → Order confirmation

Admin journey:

Admin login → Product creation → Product update → Order view → Order status update

Minimum Quality Gate

A feature is not complete until: - Happy path works - Error state works - Loading/empty state works where applicable -

Validation exists - Tests exist for important business logic - No secrets are exposed

Development Plan

Phase 0 --- Specification

1. Finalize product requirements
2. Finalize MVP scope
3. Finalize user flows
4. Finalize technical decisions
5. Finalize data model
6. Finalize API contracts

No feature implementation before this phase is sufficiently defined.

Phase 1 --- Project Foundation

- Next.js + TypeScript
- Tailwind + shadcn/ui
- ESLint/formatting
- Environment configuration
- Base folder structure
- Database connection
- Prisma setup
- Basic error handling

Phase 2 --- Authentication

- User authentication
- Session handling
- Protected routes
- Admin role authorization

Phase 3 --- Product Catalog

- Categories
- Product CRUD
- Product images
- Product listing
- Search/filter
- Product details
- Stock

Phase 4 --- Cart

- Add item
- Remove item
- Update quantity
- Server-side validation

Phase 5 --- Checkout

- Address
- Order summary
- Server-side price calculation
- Stock validation

Phase 6 --- Payments

- Razorpay integration
- Payment verification
- Webhook
- Failure handling
- Idempotency

Phase 7 --- Orders

- Create order
- Customer order history
- Order details
- Admin order management
- Status transitions

Phase 8 --- Testing & Hardening

- Unit tests
- Integration tests
- E2E tests
- Security review
- Performance review
- Accessibility review

Phase 9 --- Production

- Production database
- Environment variables
- Image storage
- Payment production configuration
- Deployment
- Monitoring/logging

Definition of Done

A feature is done only when: 1. Requirement is documented. 2. User flow is defined. 3. API/data changes are defined. 4. UI states are handled. 5. Validation exists. 6. Tests are added where appropriate. 7. Security implications are considered. 8. Code is reviewed. 9. No unrelated feature is added.

Anti-Vibe-Coding Rule

Do not invent features during implementation.

If an implementation reveals a missing requirement: 1. Stop. 2. Document the requirement/change. 3. Decide whether it belongs in MVP. 4. Update the relevant specification. 5. Then implement.

